Problems

5 min read Updated 20 May 2026 Topic 51 of 77

Chapter 24: Machine Coding — Timed Practice Problems

Total Problems: 14 **Difficulty Range:** Mid to Senior **Format:** Each problem has a time limit, required features, and bonus features. Simulate real interview conditions.

Why This Chapter Matters

Reading solutions isn't the same as building under pressure. This chapter is a set of practice problems designed to be attempted under timed conditions. No solutions are provided here — that's intentional. Build it yourself, then compare with the approaches in Chapter 20.

Rules for practice:

1. Set a timer. Do NOT go over.
2. No looking at solutions during the timer.
3. Build the working feature first, then polish.
4. After time expires, review what you built — what worked, what you'd change.
5. Try the same problem again a week later. You should be 2x faster.

Problem 1: Autocomplete Search (45 min)

Build: A search input that shows suggestions from a mock API (use `setTimeout` to simulate). Suggestions appear as a dropdown below the input.

Required:

- Debounced input (300ms)
- Loading indicator while fetching
- Click a suggestion to fill the input
- Close dropdown on Escape or click outside

Bonus: Keyboard navigation (up/down arrows, Enter to select), highlighted match text.

Evaluates: Async handling, debouncing, DOM events, component state.

Problem 2: Todo App with Filters (30 min)

Build: Add/delete/toggle todos with filter tabs (All, Active, Completed).

Required:

- Add todo on Enter
- Toggle complete with checkbox
- Delete with button
- Filter tabs that update the visible list
- Remaining count

Bonus: LocalStorage persistence, edit on double-click, "Clear completed" button.

Evaluates: State management, filtering, event handling.

Problem 3: Multi-step Form (45 min)

Build: 3-step form: Personal Info → Address → Review & Submit.

Required:

- Step indicator showing progress
- Validation per step (required fields, email format)
- Back button preserves entered data
- Review step shows all data before submit

Bonus: Disable "Next" until valid, animated step transitions, form reset after submit.

Evaluates: Form state, validation patterns, multi-component coordination.

Problem 4: Infinite Scroll Image Gallery (45 min)

Build: Grid of images that loads more when user scrolls near the bottom. Use a mock API or placeholder images.

Required:

- Grid layout (responsive — 2 columns on mobile, 4 on desktop)
- IntersectionObserver for scroll detection
- Loading skeleton while images load
- "No more images" indicator at end

Bonus: Lazy loading individual images, blur-up placeholder, lightbox on click.

Evaluates: IntersectionObserver, responsive CSS, performance awareness.

Problem 5: Star Rating with Hover Preview (30 min)

Build: 5-star rating component with hover preview and click to select.

Required:

- Stars highlight on hover (preview)
- Click sets the rating
- Display current rating value
- Works with keyboard (Tab + Enter)

Bonus: Half-star support, read-only mode, animated fill, accessible (aria-label per star).

Evaluates: Event handling, state, CSS, accessibility.

Problem 6: Kanban Board (60 min)

Build: Three columns (To Do, In Progress, Done) with draggable cards.

Required:

- Add new cards to any column
- Drag cards between columns
- Cards show title
- Visual feedback during drag

Bonus: Edit card title on click, delete card, persist to localStorage, column reordering.

Evaluates: Drag and drop, complex state, layout.

Problem 7: Countdown Timer (30 min)

Build: Set a duration (mm:ss), start countdown, show progress.

Required:

- Input for minutes and seconds

- Start, Pause, Reset buttons
- Visual countdown display (mm:ss format)
- Alert/notification when timer reaches 0

Bonus: Circular progress indicator, sound on completion, multiple timers.

Evaluates: Timer management, requestAnimationFrame vs setInterval, state.

Problem 8: Expense Tracker (45 min)

Build: Add expenses with amount, category, and date. Show total and breakdown.

Required:

- Add expense form (amount, category dropdown, date)
- List of expenses
- Total amount displayed
- Filter by category

Bonus: Category-wise pie/bar chart, date range filter, edit/delete, localStorage.

Evaluates: Form handling, derived state (totals, filtered lists), data display.

Problem 9: Chat Interface (45 min)

Build: Two-panel chat — message list on top, input at bottom. Simulate responses.

Required:

- Send messages (Enter key)
- Show sent vs received messages (different alignment/color)
- Auto-scroll to latest message
- Simulated bot response after 1-2 seconds

Bonus: Typing indicator, timestamp grouping, emoji picker, message delete.

Evaluates: List rendering, auto-scroll, async simulation, UI polish.

Problem 10: Accordion / FAQ (30 min)

Build: Collapsible FAQ sections with smooth animation.

Required:

- Click to toggle section
- Only one section open at a time (exclusive mode)
- Smooth open/close animation
- Arrow/chevron indicator

Bonus: Keyboard support, search/filter FAQ items, nested accordions, accessible (aria-expanded).

Evaluates: CSS animations, state management, accessibility.

Problem 11: Data Table with Sorting (45 min)

Build: Table of data (users or products) with sortable columns and search.

Required:

- Render table from array data
- Click column header to sort (asc → desc → none)
- Search bar to filter rows
- Sort indicator (arrow) on active column

Bonus: Pagination, column resizing, row selection checkboxes, export to CSV.

Evaluates: Array manipulation (sort, filter), table rendering, state tracking.

Problem 12: Modal with Form (30 min)

Build: Button opens a modal containing a form. Submit closes modal and shows data.

Required:

- Modal with backdrop
- Close on Escape, close on backdrop click
- Form inside with validation
- Focus trapped inside modal

Bonus: Use native `<dialog>`, animation on open/close, multiple modals stacked, accessible.

Evaluates: Focus management, event handling, form validation.

Problem 13: Memory Card Game (45 min)

Build: Grid of face-down cards. Flip two at a time to find matching pairs.

Required:

- Grid of shuffled card pairs
- Flip animation on click
- Match detection (keep matched cards face-up)
- Move counter
- Win detection (all pairs found)

Bonus: Timer, difficulty levels (4×4, 6×6), best score tracking, restart button.

Evaluates: Game logic, state management, CSS animations, array manipulation (shuffle).

Problem 14: GitHub Profile Viewer (45 min)

Build: Search GitHub usernames, display profile info and repositories.

Required:

- Search input with fetch on Enter
- Display: avatar, name, bio, follower/following counts
- List of repositories (name, stars, language)
- Loading and error states

Bonus: Pagination for repos, sort repos by stars, click repo to open in new tab, recent searches.

Evaluates: API integration, async handling, error states, UI composition.

QUICK CHECK — no marks, just for you.

1. In the Autocomplete Search problem, what debounce delay does the chapter require on the search input?

- ◯ 1000ms
- ◯ 500ms
- ◯ 300ms
- ◯ 100ms

Check answer

2. In the Data Table with Sorting problem, what sort cycle should clicking a column header follow?

- ◯ none, then descending, then ascending

- descending, then ascending, then none
- ascending, then descending, and back to ascending

Check answer

3. For the Modal with Form problem, which of these is listed as a REQUIRED feature rather than a bonus?

- Focus is trapped inside the modal
- The modal animates on open and close
- Multiple modals can be stacked
- The modal is built on the native dialog element

Check answer

Previous
JS Problems SDE3

Next
Testing fundamentals

LEARN MORE

ARTICLES & DOCS

Front-End System Design Playbook
greatfrontend.com

VIDEOS





Machine coding for frontend developers

YouTube



30 React machine-coding projects

YouTube

Hand-picked references for this lesson.